operation and notify a condition variable or populate a future, or even just sleeping for a period of time.

Whatever the reason for the waits, if you have only as many threads as there are physical processing units in the system, having blocked threads means you're wasting CPU time. The processor that would otherwise be running a blocked thread is instead doing nothing. Consequently, if you know that one of your threads is likely to spend a considerable portion of its time waiting around, you can make use of that spare CPU time by running one or more additional threads.

Consider a virus scanner application, which divides the work across threads using a pipeline. The first thread searches the filesystem for files to check and puts them on a queue. Meanwhile, another thread takes filenames from the queue, loads the files, and scans them for viruses. You know that the thread searching the filesystem for files to scan is definitely going to be I/O bound, so you make use of the "spare" CPU time by running an additional scanning thread. You'd then have one file-searching thread and as many scanning threads as there are physical cores or processors in the system. Since the scanning thread may also have to read significant portions of the files off the disk in order to scan them, it might make sense to have even more scanning threads. But at some point there'll be too many threads, and the system will slow down again as it spends more and more time task switching, as described in section 8.2.5.

As ever, this is an optimization, so it's important to measure performance before and after any change in the number of threads; the optimal number of threads will be highly dependent on the nature of the work being done and the percentage of time the thread spends waiting.

Depending on the application, it might be possible to use up this spare CPU time without running additional threads. For example, if a thread is blocked because it's waiting for an I/O operation to complete, it might make sense to use asynchronous I/O if that's available, and then the thread can perform other useful work while the I/O is performed in the background. In other cases, if a thread is waiting for another thread to perform an operation, then rather than blocking, the waiting thread might be able to perform that